

PRAKTIKUM SO

MODUL 6 - PROSES SEKUENSIAL DAN KONKUREN



Disusun Oleh :
Nama : Ganes Gemi Putra
Kelas : SE-07-02
NIM : (2311104075)

Dosen : Yudha Islami Sulistya S.Kom. M.C.s

PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY
PURWOKERTO
2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Modul 6 Praktikum Sistem Operasi membahas proses sekuensial dan konkuren pada Embedded Xinu. Pada modul ini praktikan melakukan modifikasi source code Xinu, menjalankan beberapa variasi program utama, serta mengamati perbedaan perilaku ketika fungsi dipanggil secara berurutan dibandingkan ketika dijalankan sebagai proses yang dijadwalkan oleh kernel.

Praktikum ini penting karena konsep proses merupakan bagian inti sistem operasi. Melalui percobaan pada file main.c, praktikan dapat memahami batas jumlah proses pada Xinu, penggunaan fungsi create() dan resume(), serta dampak penggunaan variabel global yang diakses bersama oleh dua proses tanpa sinkronisasi.

1.2 Tujuan Praktikum

1. Mengetahui lokasi batas maksimum proses pada source code Xinu dan mengubah nilainya menjadi 150.
2. Menjalankan program main_ex1 untuk mengamati proses sekuensial.
3. Menjalankan program main_ex2 untuk mengamati dua proses konkuren.
4. Menjalankan program main_ex3 untuk mengamati penggunaan satu fungsi dengan parameter berbeda pada dua proses konkuren.
5. Membuat program produser-konsumer sederhana dan menganalisis hasil akses variabel global bersama.

1.3 Manfaat Praktikum

Praktikum ini bermanfaat untuk melatih pemahaman praktikan terhadap struktur source code Xinu, mekanisme dasar pembentukan proses, penjadwalan proses, serta permasalahan konkurensi seperti race condition. Hasil praktikum menjadi dasar untuk mempelajari sinkronisasi proses pada modul berikutnya.

BAB II DASAR TEORI

2.1 Embedded Xinu

Embedded Xinu adalah sistem operasi kecil yang umum dipakai sebagai media pembelajaran sistem operasi. Xinu menyediakan kernel sederhana, manajemen proses, penjadwalan, shell, dan struktur source code yang cukup ringkas sehingga mudah dipelajari secara langsung melalui eksperimen.

2.2 Proses Sekuensial

Proses sekuensial adalah eksekusi instruksi secara berurutan. Jika sebuah fungsi dipanggil langsung di dalam main() dan fungsi tersebut belum selesai, maka fungsi berikutnya belum akan dijalankan. Pada kondisi ini alur program mengikuti urutan pemanggilan fungsi.

2.3 Proses Konkuren

Proses konkuren adalah keadaan ketika beberapa proses berada dalam sistem dan dijadwalkan bergantian oleh CPU. Pada Xinu, proses baru dapat dibuat menggunakan create()

dan diaktifkan dengan `resume()`. Walaupun CPU menjalankan satu instruksi pada satu waktu, hasilnya tampak seperti dua proses berjalan bersama.

2.4 Fungsi `create()` dan `resume()`

Fungsi `create()` digunakan untuk membuat process control block baru beserta stack dan parameter awalnya. Setelah proses berhasil dibuat, fungsi `resume()` dipakai untuk mengubah status proses menjadi siap berjalan sehingga dapat dipilih oleh scheduler.

2.5 Race Condition

Race condition terjadi ketika dua proses atau lebih mengakses data bersama tanpa mekanisme sinkronisasi yang tepat. Akibatnya hasil pembacaan atau penulisan dapat berubah-ubah tergantung urutan eksekusi scheduler. Dalam praktikum ini, variabel global `n` dipakai bersama oleh produser dan konsumen untuk memperlihatkan efek tersebut.

BAB III LANGKAH KERJA

3.1 Persiapan

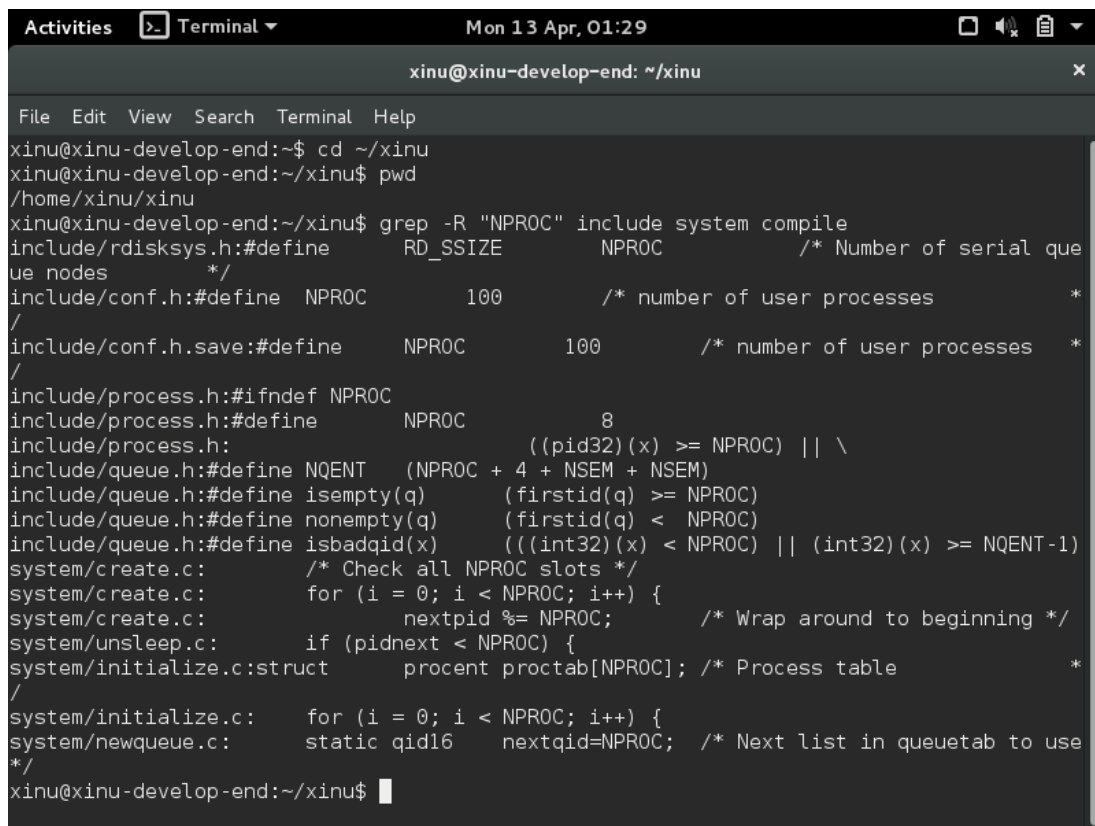
Tahap awal praktikum dilakukan dengan menyiapkan direktori Xinu, source code utama, serta terminal untuk proses kompilasi dan pengujian. Pengerjaan jurnal nomor 2 sampai nomor 5 mengikuti catatan praktikum yang diberikan kepada praktikan.

3.2 Langkah Persiapan Pengerjaan Jurnal Nomor 2-5

1. Buka direktori `xinu/system`.
2. Backup file bernama `main.c` ke folder Home dan jangan menaruh backup pada folder yang sama.
3. Setelah backup, buka kembali file `main.c` yang terdapat pada direktori `xinu/system`.
4. Masukkan kode sesuai soal pada nomor 2, 3, 4, dan 5 secara bergantian.
5. Setelah selesai melakukan perubahan, simpan file, lakukan compile, lalu jalankan Xinu seperti pada modul-modul sebelumnya.
6. Lakukan screenshot kode program dan output akhir pada Xinu sebagai bukti pengerjaan.

3.3 Pelaksanaan Nomor 1

Untuk nomor 1, praktikan mencari konstanta pembatas jumlah proses menggunakan perintah pencarian terhadap simbol `NPROC`. Setelah lokasi file ditemukan, praktikan membuka `include/conf.h`, mencatat nilai awal maksimum proses, mengubahnya menjadi 150, lalu melakukan kompilasi ulang agar perubahan diterapkan ke sistem.

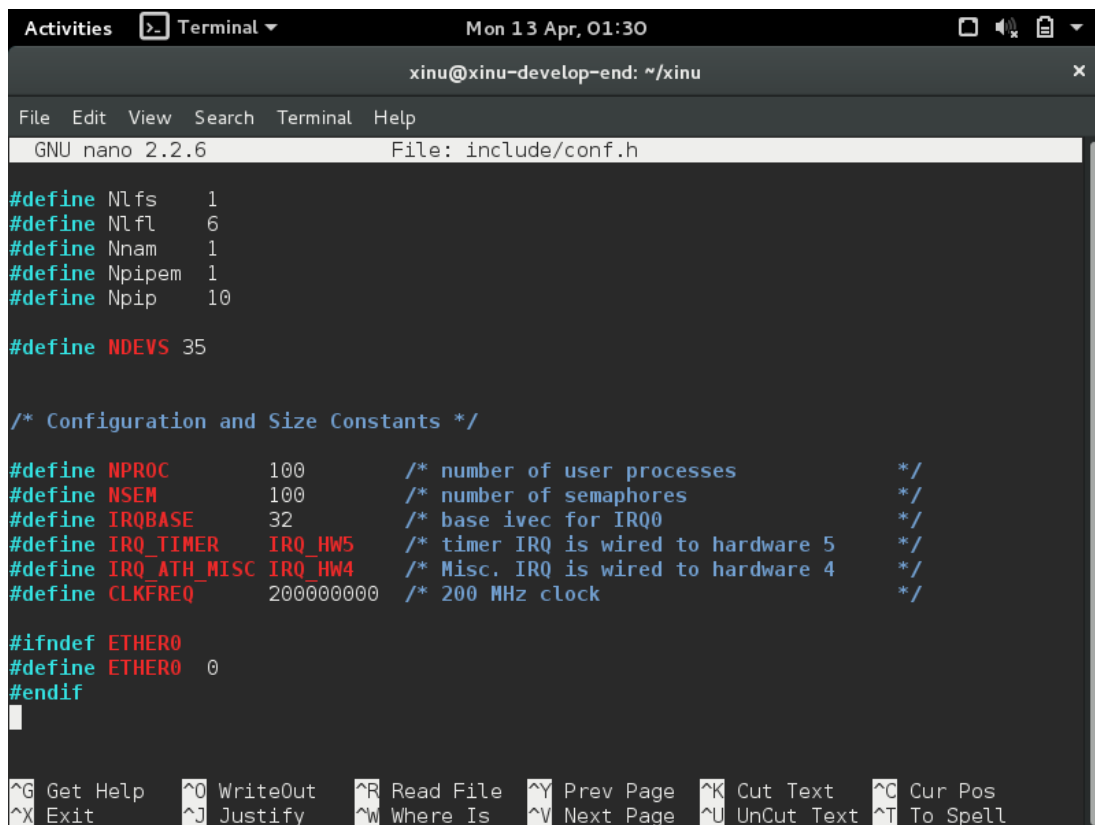


```

xinu@xinu-develop-end: ~/xinu
File Edit View Search Terminal Help
xinu@xinu-develop-end:~$ cd ~/xinu
xinu@xinu-develop-end:~/xinu$ pwd
/home/xinu/xinu
xinu@xinu-develop-end:~/xinu$ grep -R "NPROC" include system compile
include/rdisksys.h:#define      RD_SSIZE      NPROC      /* Number of serial que
ue nodes      */
include/conf.h:#define  NPROC      100      /* number of user processes      *
/
include/conf.h.save:#define      NPROC      100      /* number of user processes      *
/
include/process.h:#ifndef NPROC
include/process.h:#define      NPROC      8
include/process.h:      ((pid32)(x) >= NPROC) || \
include/queue.h:#define NQENT      (NPROC + 4 + NSEM + NSEM)
include/queue.h:#define isempty(q)      (firstid(q) >= NPROC)
include/queue.h:#define nonempty(q)      (firstid(q) < NPROC)
include/queue.h:#define isbadqid(x)      (((int32)(x) < NPROC) || (int32)(x) >= NQENT-1)
system/create.c:      /* Check all NPROC slots */
system/create.c:      for (i = 0; i < NPROC; i++) {
system/create.c:          nextpid %= NPROC;      /* Wrap around to beginning */
system/unsleep.c:      if (pidnext < NPROC) {
system/initialize.c:struct      procent proctab[NPROC]; /* Process table      *
/
system/initialize.c:      for (i = 0; i < NPROC; i++) {
system/newqueue.c:      static qid16      nextqid=NPROC; /* Next list in queuetab to use
*/
xinu@xinu-develop-end:~/xinu$

```

Gambar 3.1 Hasil pencarian simbol NPROC pada source code Xinu



```

xinu@xinu-develop-end: ~/xinu
File Edit View Search Terminal Help
GNU nano 2.2.6      File: include/conf.h
#define Nlfs      1
#define Nlfl      6
#define Nnam      1
#define Npipem      1
#define Npip      10

#define NDEVS      35

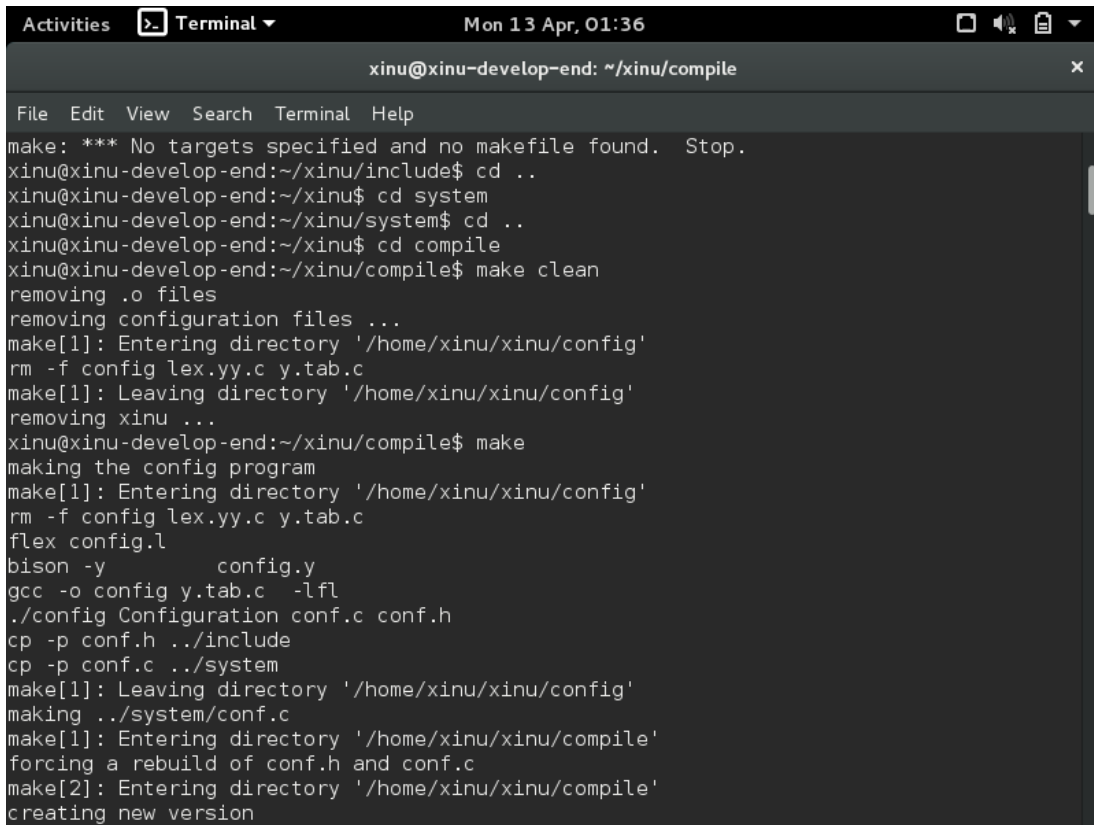
/* Configuration and Size Constants */

#define NPROC      100      /* number of user processes      */
#define NSEM      100      /* number of semaphores      */
#define IRQBASE      32      /* base ivec for IRQ0      */
#define IRQ_TIMER      IRQ_HW5      /* timer IRQ is wired to hardware 5      */
#define IRQ_ATH_MISC      IRQ_HW4      /* Misc. IRQ is wired to hardware 4      */
#define CLKFREQ      200000000      /* 200 MHz clock      */

#ifndef ETHERO
#define ETHERO      0
#endif

```

Gambar 3.2 Tampilan file include/conf.h sebelum nilai NPROC diubah



```

xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
make: *** No targets specified and no makefile found. Stop.
xinu@xinu-develop-end:~/xinu/include$ cd ..
xinu@xinu-develop-end:~/xinu$ cd system
xinu@xinu-develop-end:~/xinu/system$ cd ..
xinu@xinu-develop-end:~/xinu$ cd compile
xinu@xinu-develop-end:~/xinu/compile$ make clean
removing .o files
removing configuration files ...
make[1]: Entering directory '/home/xinu/xinu/config'
rm -f config lex.yy.c y.tab.c
make[1]: Leaving directory '/home/xinu/xinu/config'
removing xinu ...
xinu@xinu-develop-end:~/xinu/compile$ make
making the config program
make[1]: Entering directory '/home/xinu/xinu/config'
rm -f config lex.yy.c y.tab.c
flex config.l
bison -y          config.y
gcc -o config y.tab.c -lfl
./config Configuration conf.c conf.h
cp -p conf.h ../include
cp -p conf.c ../system
make[1]: Leaving directory '/home/xinu/xinu/config'
making ../system/conf.c
make[1]: Entering directory '/home/xinu/xinu/compile'
forcing a rebuild of conf.h and conf.c
make[2]: Entering directory '/home/xinu/xinu/compile'
creating new version

```

Gambar 3.3 Proses kompilasi ulang Xinu setelah konfigurasi diperbarui

3.4 Pelaksanaan Nomor 2-5

Pada nomor 2 sampai nomor 5, file main.c dibuka kembali dari direktori xinu/system. Praktikan mengganti isi file sesuai source code masing-masing soal, menyimpan perubahan, melakukan make clean dan make dari folder xinu/compile, kemudian menjalankan Xinu untuk mengamati output program. Setiap soal didokumentasikan melalui screenshot kode dan screenshot output akhir.

BAB IV HASIL DAN PEMBAHASAN

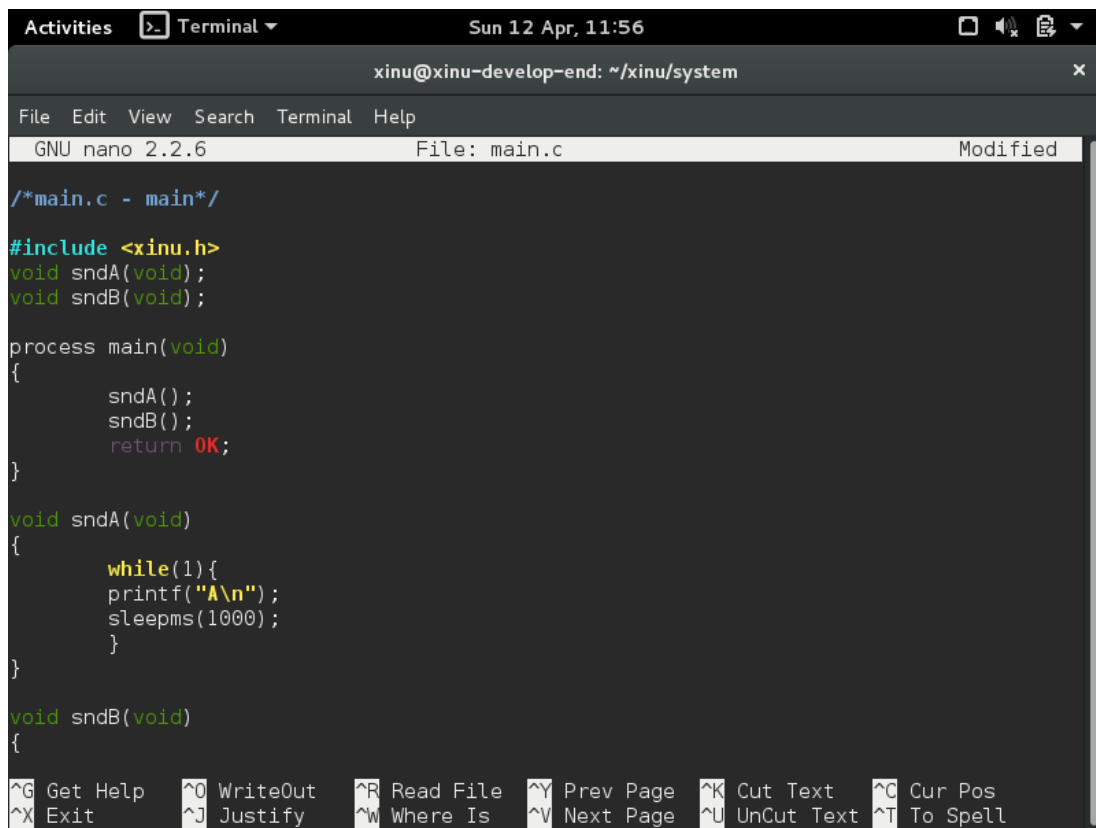
4.1 Analisis Nomor 1: Batas Maksimum Proses pada Xinu

Hasil pencarian menunjukkan bahwa batas jumlah proses pada source code Xinu berada pada file include/conf.h melalui konstanta NPROC. Berdasarkan hasil sebelum pengubahan, nilai maksimum proses yang terkonfigurasi adalah 100. Pada praktikum ini nilai tersebut kemudian diubah menjadi 150 sesuai instruksi soal.

Setelah perubahan selesai dilakukan, Xinu dikompilasi ulang agar konfigurasi baru diterapkan. Perubahan ini menunjukkan bahwa jumlah proses pada Xinu tidak hanya dibatasi oleh perangkat keras, tetapi juga dapat diatur melalui konstanta konfigurasi pada source code.

4.2 Analisis Nomor 2: main_ex1

Source code nomor 2 memanggil fungsi sndA() dan sndB() secara langsung dari main(). Namun, fungsi sndA() berisi while(1) yang tidak pernah berhenti sehingga alur program selalu berada di dalam sndA(). Akibatnya, fungsi sndB() tidak pernah sempat dieksekusi.



```

/*main.c - main*/
#include <xinu.h>
void sndA(void);
void sndB(void);

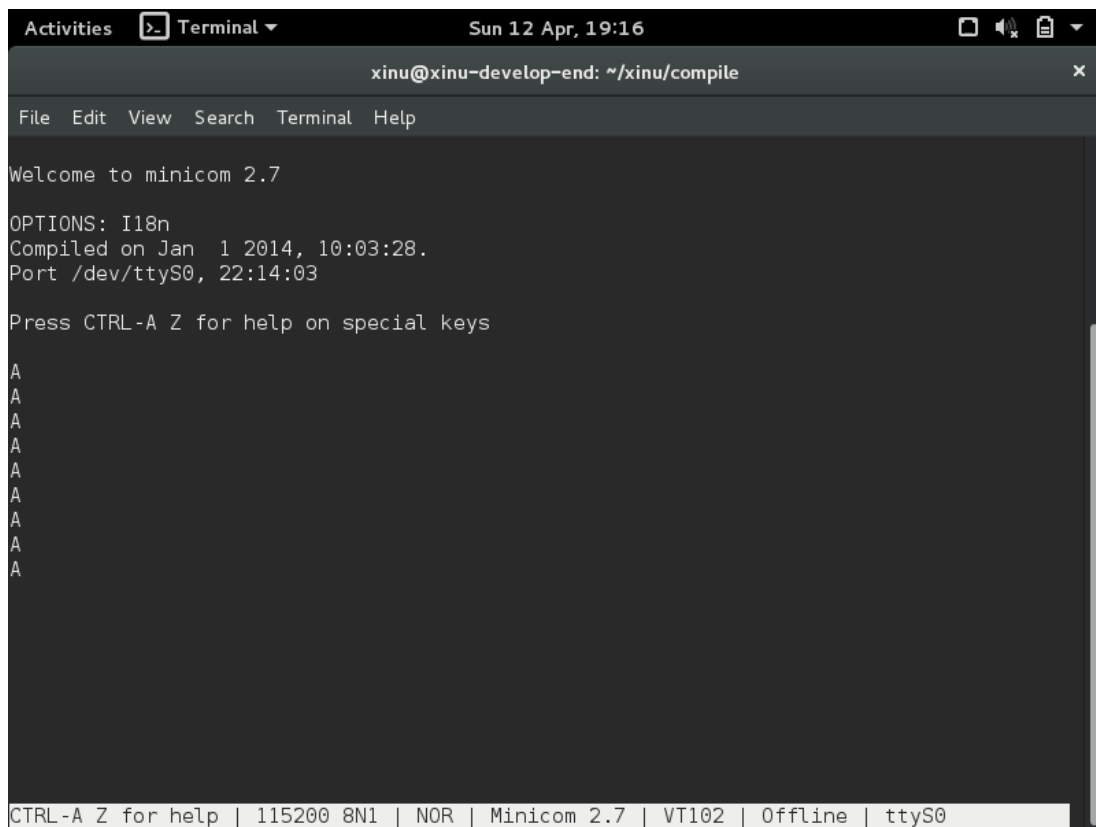
process main(void)
{
    sndA();
    sndB();
    return OK;
}

void sndA(void)
{
    while(1){
        printf("A\n");
        sleeps(1000);
    }
}

void sndB(void)
{

```

Gambar 4.1 Source code nomor 2 (main_ex1) pada file main.c



```

Welcome to minicom 2.7

OPTIONS: I18n
Compiled on Jan  1 2014, 10:03:28.
Port /dev/ttyS0, 22:14:03

Press CTRL-A Z for help on special keys

A
A
A
A
A
A
A
A
A
A
A

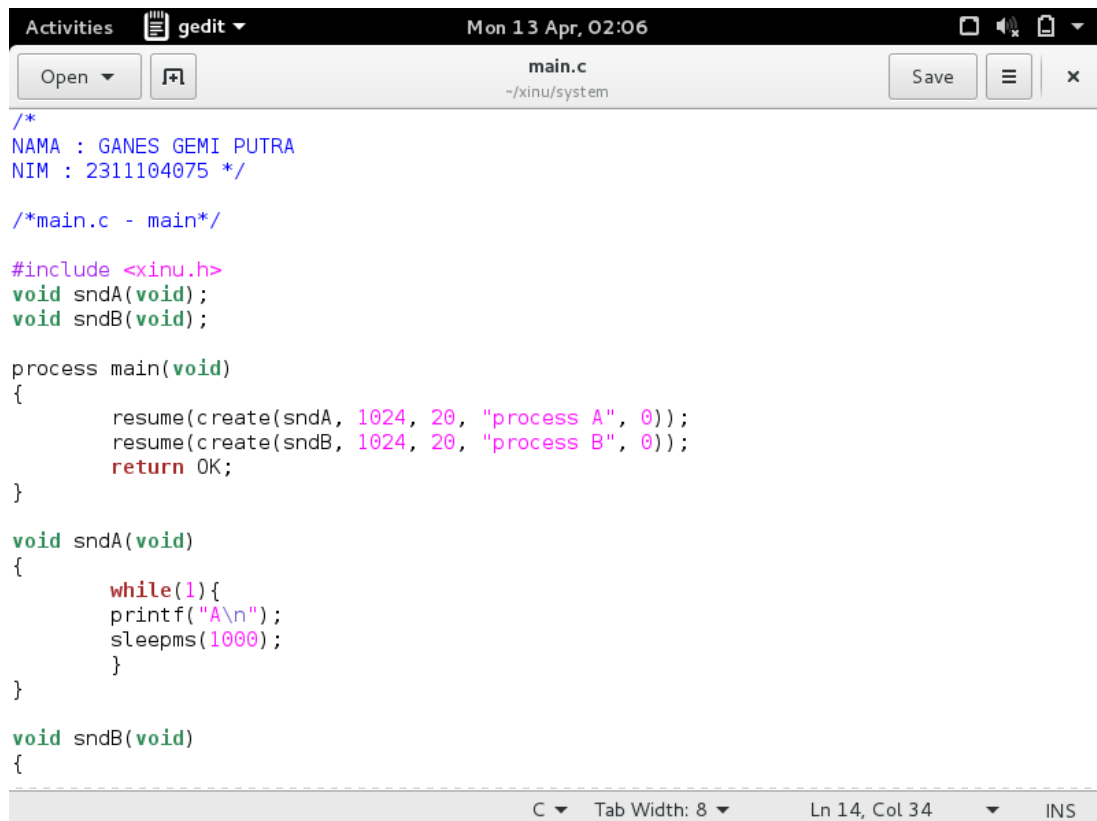
```

Gambar 4.2 Output nomor 2 yang hanya menampilkan huruf A

Output yang diamati berupa huruf A yang terus berulang. Hal ini membuktikan bahwa program berjalan secara sekuensial karena fungsi berikutnya baru akan dijalankan jika fungsi sebelumnya telah selesai.

4.3 Analisis Nomor 3: main_ex2

Pada nomor 3, dua proses dibuat menggunakan `create()` lalu diaktifkan menggunakan `resume()`. Kedua proses tersebut masing-masing menjalankan `sndA()` dan `sndB()`. Dengan demikian, scheduler Xinu dapat menjalankan kedua proses secara konkuren.



```
Activities gedit Mon 13 Apr, 02:06
main.c
~/xinu/system

/*
NAMA : GANES GEMI PUTRA
NIM : 2311104075 */

/*main.c - main*/

#include <xinu.h>
void sndA(void);
void sndB(void);

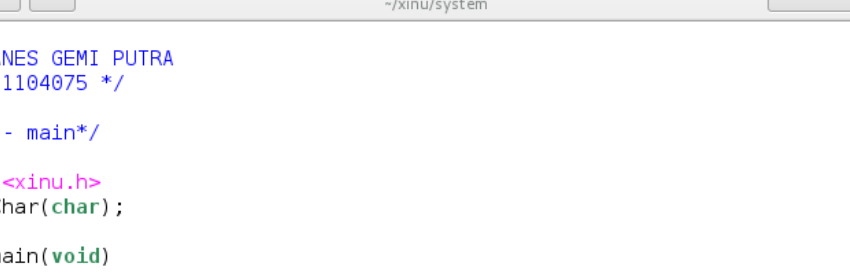
process main(void)
{
    resume(create(sndA, 1024, 20, "process A", 0));
    resume(create(sndB, 1024, 20, "process B", 0));
    return OK;
}

void sndA(void)
{
    while(1){
        printf("A\n");
        sleepms(1000);
    }
}

void sndB(void)
{

```

Gambar 4.3 Source code nomor 3 (main_ex2) yang menggunakan `create()` dan `resume()`



```
/*
NAMA : GANES GEMI PUTRA
NIM : 2311104075 */

/*main.c - main*/

#include <xinu.h>
void sndChar(char);

process main(void)
{
    resume(create(sndChar, 1024, 20, "Send A", 1, 'A'));
    resume(create(sndChar, 1024, 20, "Send B", 1, 'B'));
    return OK;
}

void sndChar(char c)
{
    while(1){
        printf("%c \n", c);
        sleeps(1000);
    }
}
```

Gambar 4.5 Source code nomor 4 (main ex3) menggunakan fungsi sndChar(char c)

```
xinu@xinu-develop-end: ~/xinu/compile
```

File Edit View Search Terminal Help

Xinu for Vbox -- version #114 (xinu) Mon 13 Apr 05:30:24 EDT 2026

Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
4447448 bytes of free memory. Free list:
[0x0015E320 to 0x0009FFF7]
[0x00100000 to 0x005FBFFF]
115369 bytes of Xinu code.
[0x00100000 to 0x0011C2A8]
148456 bytes of data.
[0x0011FEE0 to 0x001442C7]

Obtained IP address 10.0.3.15 (0x0a00030f)

A
B
A
B
A
B
A
B
A
B
|

```
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

Gambar 4.6 Output nomor 4 yang menampilkan A dan B dari satu fungsi yang sama

Hasil output tetap menampilkan A dan B secara bergantian. Perbedaannya, pada nomor 4 kedua proses tidak lagi memanggil dua fungsi yang berbeda, melainkan satu fungsi yang sama dengan argumen karakter yang berbeda.

4.5 Analisis Nomor 5: Produser dan Konsumer

Pada nomor 5 dibuat dua proses, yaitu produser dan konsumer. Proses produser menaikkan nilai variabel global n dari 0 sampai 1000, sedangkan proses konsumer membaca dan menampilkan nilai n sebanyak 1000 kali. Kedua proses tersebut dijalankan tanpa mekanisme sinkronisasi.



```
Activities gedit Mon 13 Apr, 03:16
main.c
~/xinu/system

/*
NAMA : GANES GEMI PUTRA
NIM : 2311104075 */

/*main.c - main*/

#include <xinu.h>

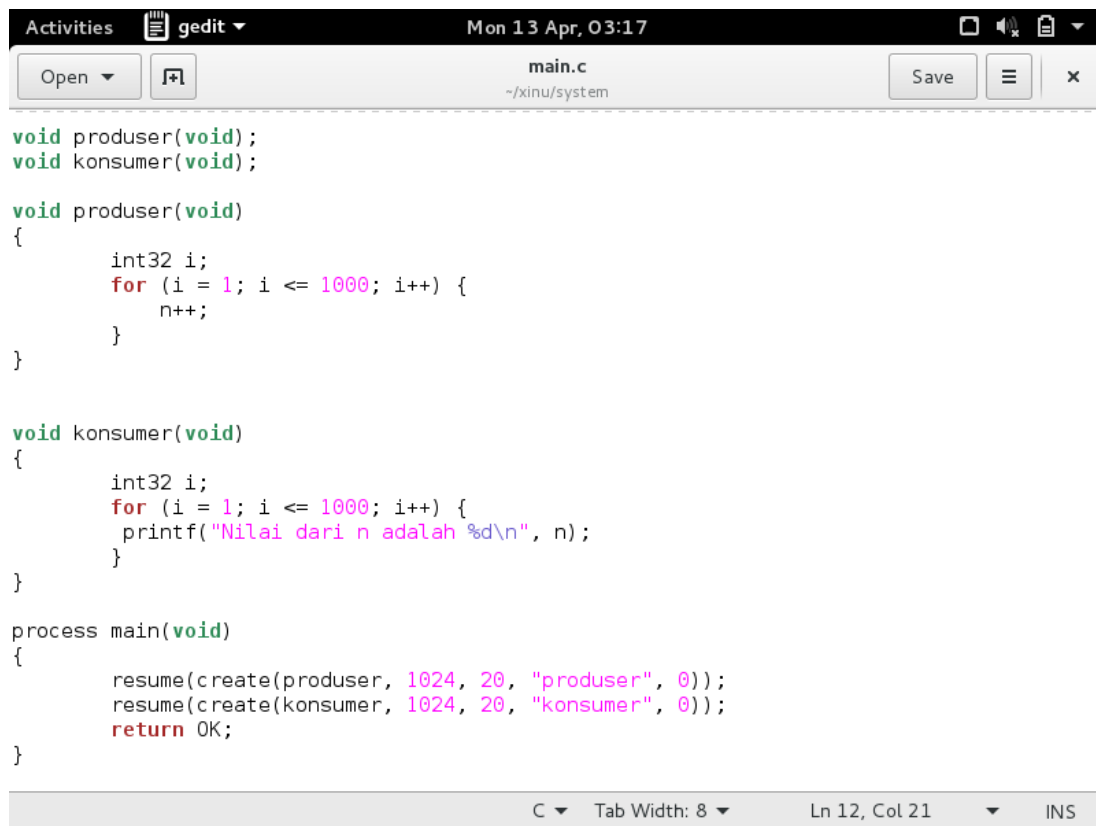
int32 n = 0;

void produser(void);
void konsumer(void);

void produser(void)
{
    int32 i;
    for (i = 1; i <= 1000; i++) {
        n++;
    }
}

void konsumer(void)
{
    int32 i;
    for (i = 1; i <= 1000; i++) {
        printf("Nilai dari n adalah %d\n", n);
    }
}
```

Gambar 4.7 Bagian atas source code nomor 5 yang memuat deklarasi variabel global dan fungsi produser



```

void produser(void);
void konsumer(void);

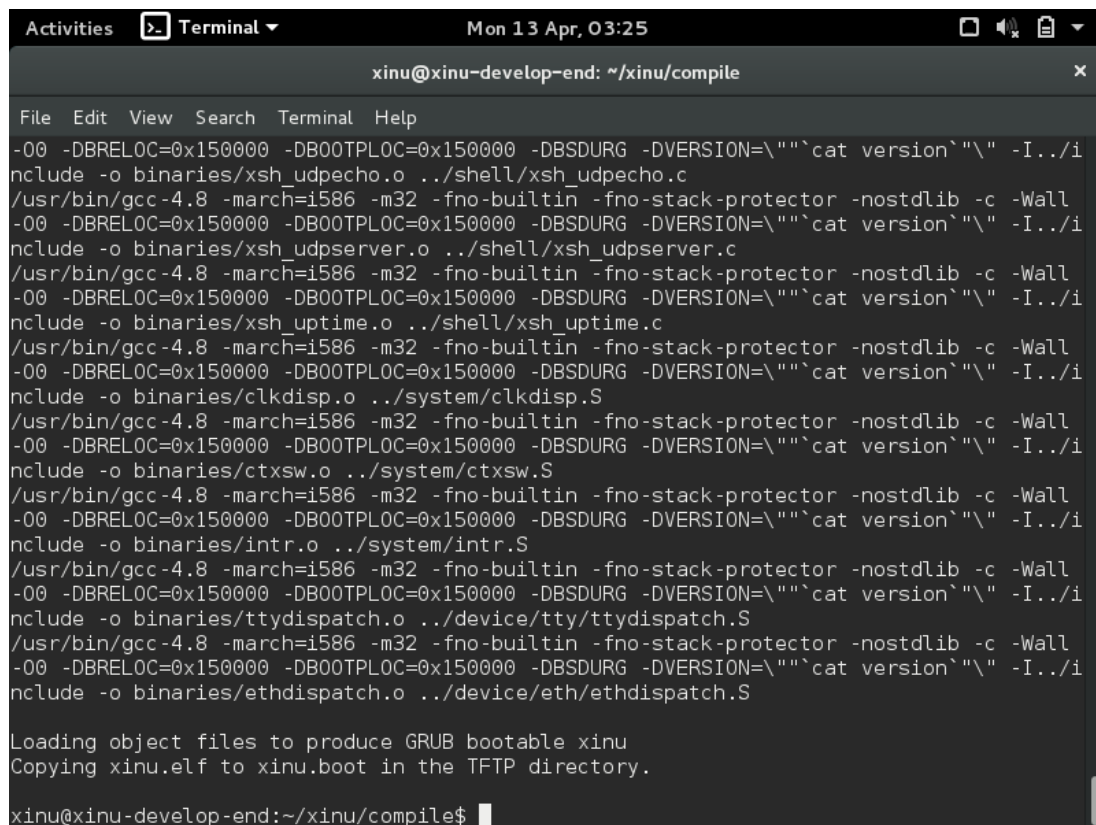
void produser(void)
{
    int32 i;
    for (i = 1; i <= 1000; i++) {
        n++;
    }
}

void konsumer(void)
{
    int32 i;
    for (i = 1; i <= 1000; i++) {
        printf("Nilai dari n adalah %d\n", n);
    }
}

process main(void)
{
    resume(create(produser, 1024, 20, "produser", 0));
    resume(create(konsumer, 1024, 20, "konsumer", 0));
    return OK;
}

```

Gambar 4.8 Bagian bawah source code nomor 5 yang memuat fungsi konsumer dan main()



```

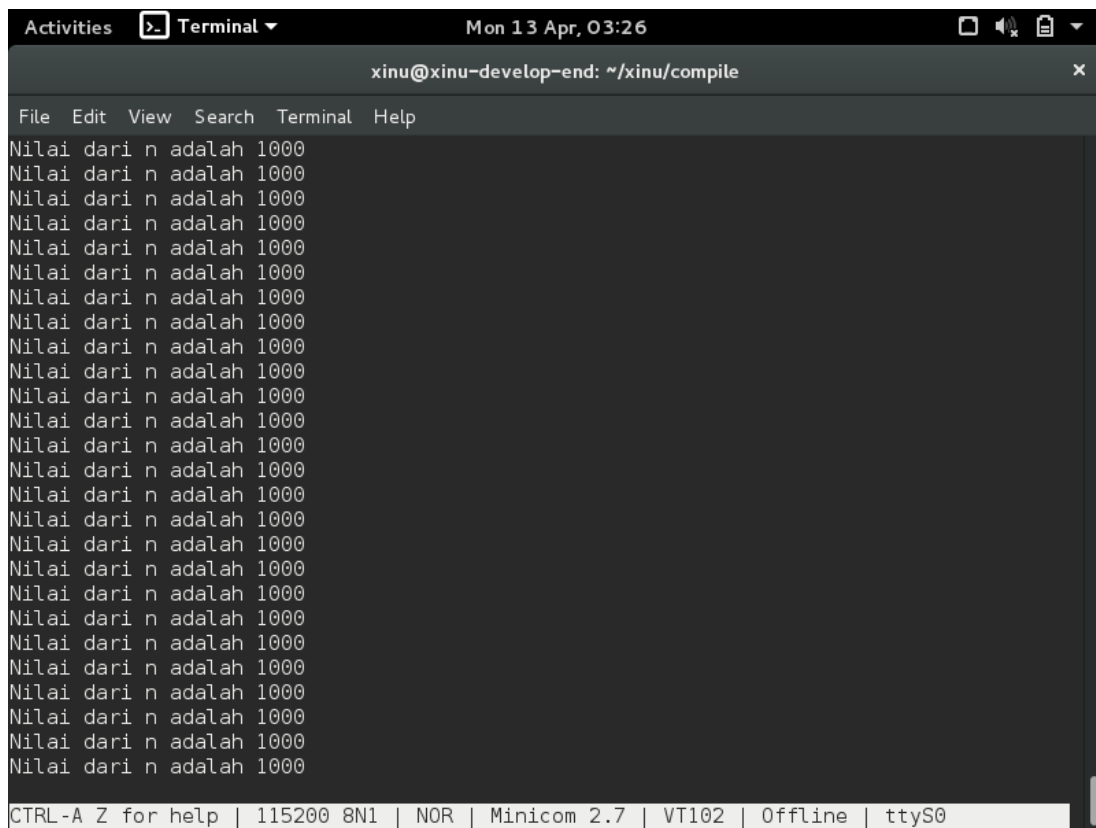
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`" -I../i
nclude -o binaries/clkdsp.o ../system/clkdsp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$

```

Gambar 4.9 Proses kompilasi program nomor 5



Gambar 4.10 Output nomor 5 yang menampilkan nilai variabel n

Pada hasil praktikum, konsumer menampilkan teks "Nilai dari n adalah 1000" berulang kali. Hasil ini tetap sesuai dengan konsep konkurensi. Kondisi tersebut menunjukkan bahwa pada saat konsumer mulai melakukan pembacaan, produser kemungkinan sudah lebih dahulu menyelesaikan seluruh operasi penambahan hingga n bernilai 1000.

Meskipun pada eksekusi ini nilai yang terbaca cenderung tetap, program tetap mengandung race condition karena dua proses mengakses variabel global yang sama tanpa semaphore, mutex, atau mekanisme sinkronisasi lain. Pada eksekusi berbeda, hasil dapat berubah-ubah, berulang, meloncat, atau tidak sesuai intuisi awal.

4.6 Pembahasan Umum

Seluruh percobaan pada Modul 6 memperlihatkan perbedaan nyata antara eksekusi sekuensial dan konkuren pada Xinu. Nomor 2 menunjukkan efek pemanggilan fungsi secara langsung, sedangkan nomor 3 dan nomor 4 menunjukkan bagaimana `create()` dan `resume()` memungkinkan beberapa proses berjalan secara bergantian. Nomor 5 menegaskan bahwa konkurensi juga memunculkan permasalahan baru ketika data dipakai bersama tanpa sinkronisasi.

BAB V PENUTUP

5.1 Kesimpulan

Berdasarkan hasil praktikum, batas maksimum proses pada source code Xinu ditemukan pada konstanta `NPROC` di file `include/conf.h` dengan nilai awal 100, kemudian berhasil diubah menjadi 150. Program `main_ex1` menunjukkan proses sekuensial karena output hanya

menampilkan huruf A. Program `main_ex2` dan `main_ex3` menunjukkan proses konkuren karena output menampilkan huruf A dan B secara bergantian. Program produser-konsumer pada nomor 5 memperlihatkan bahwa akses variabel global bersama tanpa sinkronisasi dapat menimbulkan race condition.